Task-2 Algorithms and Problem Solving

Solve algorithmic problems demonstrating sorting, searching, and greedy/dynamic approaches.

Requirements

- Implement at least 3 problems (e.g., quicksort/mergesort, binary search, knapsack or interval scheduling)
- Include test cases and measure runtime for different input sizes

Deliverables

1. Source files and test inputs
2. Report with algorithm choice and complexity

Binary Search

Objective

The objective of this task is to implement searching, sorting, and greedy algorithms using the C programming language and analyze their efficiency using time complexity.

---

1. Binary Search

Aim

To search an element in a sorted array using Binary Search algorithm.

Algorithm

1. Start with middle element.
2. Compare target element with middle element.
3. If equal, element found.
4. If smaller, search left half.

5. If larger, search right half.

Time Complexity

O(log n)

Run Debug Stop Share Saved { } Beautify

main.c

```c
#include <stdio.h>

int main()
{
    int arr[100], n, i;
    int search, low, high, mid;
    int found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter sorted array elements:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &search);

    low = 0;
    high = n - 1;

    // Binary Search
    // Time Complexity: O(log n)

```

```c
    while(low <= high)
    {
        mid = (low + high) / 2;

        if(arr[mid] == search)
        {
            found = 1;
            break;
        }
        else if(arr[mid] < search)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    if(found)
    {
        printf("Element found at position %d\n", mid + 1);
    }
    else
    {
        printf("Element not found\n");
    }
```

```c
        printf("Element not found\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 4
Enter sorted array elements:
1 3 4 6
Enter element to search: 4
Element found at position 3


...Program finished with exit code 0
```

Explanation

Binary Search works on sorted arrays by repeatedly dividing the search space into two halves. It is more efficient than linear search.

2. Merge Sort

Aim

To sort array elements using Merge Sort algorithm.

Algorithm

1. Divide array into two halves.
2. Recursively sort both halves.
3. Merge sorted halves.

Time Complexity

O(n log n)

Run Debug Stop Share Saved { } Beautify

main.c

```c
#include <stdio.h>

void merge(int arr[], int low, int mid, int high)
{
    int i, j, k;
    int n1 = mid - low + 1;
    int n2 = high - mid;

    int left[n1], right[n2];

    for(i = 0; i < n1; i++)
        left[i] = arr[low + i];

    for(j = 0; j < n2; j++)
        right[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = low;

    while(i < n1 && j < n2)
    {
        if(left[i] <= right[j])
        {
            arr[k] = left[i];
            i++;
        }
```

```c
        else
        {
            arr[k] = right[j];
            j++;
        }

        k++;
    }

    while(i < n1)
    {
        arr[k] = left[i];
        i++;
        k++;
    }

    while(j < n2)
    {
        arr[k] = right[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int low, int high)
{
    // Time Complexity: O(n log n)
```

```c
}

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);

    printf("Sorted Array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 4
Enter array elements:
2 3 4 7
Sorted Array:
2 3 4 7

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation

Merge Sort follows divide-and-conquer approach by dividing arrays into smaller parts and merging them after sorting.

3. Knapsack Problem

Aim

To solve Knapsack problem using greedy approach.

Time Complexity

O(n log n)

Run Debug Stop Share Saved { } Beautify

main.c

```c
#include <stdio.h>

int main()
{
    int n, i, j;
    int weight[10], profit[10];
    int capacity;
    int ratioIndex;
    float ratio[10];
    float totalProfit = 0;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter profits of items:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &profit[i]);
    }

    printf("Enter weights of items:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &weight[i]);
    }
```

Run Debug Stop Share Saved { } Beautify

main.c

```c
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    // Calculate profit/weight ratio
    for(i = 0; i < n; i++)
    {
        ratio[i] = (float)profit[i] / weight[i];
    }

    // Greedy approach
    // Time Complexity: O(n²)

    printf("\nSelected Items:\n");

    while(capacity > 0)
    {
        ratioIndex = -1;

        for(i = 0; i < n; i++)
        {
            if(ratio[i] > 0)
            {
                if(ratioIndex == -1 || ratio[i] > ratio[ratioIndex])
                {
                    ratioIndex = i;
```

```c
        if(ratioIndex == -1)
        {
            break;
        }

        if(weight[ratioIndex] <= capacity)
        {
            printf("Item %d selected completely\n", ratioIndex + 1);

            capacity -= weight[ratioIndex];
            totalProfit += profit[ratioIndex];
        }
        else
        {
            float fraction = (float)capacity / weight[ratioIndex];

            printf("Item %d selected partially\n", ratioIndex + 1);

            totalProfit += profit[ratioIndex] * fraction;

            capacity = 0;
        }

        ratio[ratioIndex] = 0;
    }

    printf("\nMaximum Profit = %.2f\n", totalProfit);
```

```c
    printf("\nMaximum Profit = %.2f\n", totalProfit);

    return 0;
}
```

Output:

```
Enter number of items: 4
Enter profits of items:
4 5 7 9
Enter weights of items:
3 7 5 9
Enter knapsack capacity: 40

Selected Items:
```

```
Selected Items:
Item 3 selected completely
Item 1 selected completely
Item 4 selected completely
Item 2 selected completely

Maximum Profit = 25.00
```

Conclusion

In this task, searching, sorting, and greedy algorithms were successfully implemented using C programming language. The performance and efficiency of algorithms were analyzed using time complexity concepts.